



GUÍA DE IMPLEMENTACIÓN - StyleStock v2.0



RESUMEN DE LO GENERADO

He creado la estructura completa del sistema adaptado con:



Base de Datos

- `create_tables_v2.sql` - Schema completo con:
 - Modelo flexible de productos con atributos dinámicos
 - Sistema de precios mayorista/minorista
 - Ventas con múltiples tipos de comprobante
 - Pagos combinados
 - Auditoría completa



Modelos Java

- `Categoria.java`
- `Atributo.java` y `ValorAtributo.java`
- `Producto.java` (antes Modelo)
- `Variante.java` con atributos dinámicos
- `Ciente.java` con tipos de cliente
- `Venta.java` (antes Factura)
- `DetalleVenta.java`, `PagoVenta.java`
- `MetodoPago.java`, `TipoCliente.java`



DAOs

- `CategoriaDAO.java`
- `ProductoDAO.java`
- `VarianteDAO.java`
- `VentaDAO.java`
- `CienteDAO.java`



Services

- `ProductoService.java`
- `VarianteService.java`

- `VentaService.java`
- `ClienteService.java`
- `CategoriaService.java`

✓ Controllers

- `VentaController.java` - Controlador completo de ventas
- `MainController.java` - Controlador principal actualizado

✓ Vistas FXML

- `venta-view.fxml` - Interfaz completa de ventas
- `main-view.fxml` - Menú principal actualizado

📁 ARCHIVOS QUE FALTAN CREAR

1. Modelos Faltantes

Debes hacer los modelos públicos (quitar `class` y poner `public class`) en:

- `modelos_ventas.java` contiene: Cliente, Venta, DetalleVenta, MetodoPago, PagoVenta
- Sepáralos en archivos individuales

2. DAOs y Services Faltantes

Crear:

- `AtributoDAO.java`
- `ValorAtributoDAO.java`
- `MetodoPagoDAO.java`
- `TipoClienteDAO.java`

Y sus respectivos Services:

- `AtributoService.java`
- `MetodoPagoService.java`
- `TipoClienteService.java`

3. Controllers Faltantes

- `ProductoController.java` - Adaptado para el nuevo modelo

- VarianteController.java - Nuevo, para gestionar variantes
- ClienteController.java - Actualizado
- CategoriaController.java - Nuevo
- ConfiguracionController.java - Nuevo
- ReportesController.java - Nuevo

4. Vistas FXML Faltantes

- producto-view.fxml - Adaptada
- variante-view.fxml - Nueva
- cliente-view.fxml - Actualizada
- categoria-view.fxml - Nueva
- configuracion-view.fxml - Nueva
- reportes-view.fxml - Nueva



PASOS PARA COMPLETAR

PASO 1: Actualizar Base de Datos

```
bash
```

```
# 1. Eliminar base de datos anterior
```

```
rm ~/style-stock/style_stock.db
```

```
# 2. El sistema creará automáticamente la nueva BD al iniciar
```

```
# usando el archivo create_tables_v2.sql
```

PASO 2: Reemplazar archivos en el proyecto

Reemplaza los siguientes archivos del proyecto original:

src/main/resources/sql/create_tables.sql

→ con create_tables_v2.sql

src/main/java/com/style/stock/Main.java

→ con Main.java actualizado

src/main/java/com/style/stock/controller/MainController.java

→ con MainController.java actualizado

src/main/resources/fxml/main-view.fxml

→ con main-view.fxml actualizado

PASO 3: Agregar nuevos archivos

Modelos (crear en `src/main/java/com/style/stock/model/`):

- Separar las clases de `modelos_ventas.java` en archivos individuales
- Agregar los nuevos: `Atributo.java`, `ValorAtributo.java`, `VarianteAtributo.java`, `TipoCliente.java`

DAOs (crear en `src/main/java/com/style/stock/dao/`):

- `CategoriaDAO.java` ✓
- `ProductoDAO.java` ✓ (está en `dao_producto_variante.java`)
- `VarianteDAO.java` ✓ (está en `dao_producto_variante.java`)
- `VentaDAO.java` ✓ (está en `dao_venta_cliente.java`)
- `ClienteDAO.java` ✓ (está en `dao_venta_cliente.java`)
- Crear los faltantes mencionados arriba

Services (crear en `src/main/java/com/style/stock/service/`):

- Separar las clases de `services_principales.java`
- Crear los faltantes

Controllers (crear en `src/main/java/com/style/stock/controller/`):

- `VentaController.java` ✓
- Crear los demás

Vistas FXML (crear en `src/main/resources/fxml/`):

- `venta-view.fxml` ✓
- Crear las demás

PASO 4: Adaptaciones necesarias

En VarianteService:

Agregar el método `buscarPorTexto`:

```
java

public List<Variante> buscarPorTexto(String termino) throws DataAccessException {
    if (termino == null || termino.trim().isEmpty()) {
        return new ArrayList<>();
    }

    // Buscar por SKU o en descripción del producto
    List<Variante> resultados = new ArrayList<>();

    // Primero buscar por SKU exacto
    try {
        Variante porSku = varianteDAO.findBySku(termino);
        if (porSku != null) {
            resultados.add(porSku);
            return resultados;
        }
    } catch (DataAccessException e) {
        // Continuar con búsqueda parcial
    }

    // Buscar parcialmente
    // TODO: Implementar búsqueda en DAO
    return resultados;
}
```

Actualizar pom.xml (ya debería estar correcto):

```
xml

<properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <javafx.version>21.0.1</javafx.version>
</properties>
```

PRIORIDADES DE IMPLEMENTACIÓN

FASE 1: Core Funcional (CRÍTICO)

1.  Base de datos
2.  Modelos básicos
3.  DAOs principales
4.  Services principales
5.  VentaController y vista
6.  Separar clases consolidadas en archivos individuales

FASE 2: Gestión de Datos (IMPORTANTE)

7. Controllers de Producto, Variante, Cliente
8. Vistas correspondientes
9. Validaciones completas

FASE 3: Configuración (IMPORTANTE)

10. Controller y vista de Categorías
11. Controller y vista de Atributos
12. Controller de Configuración

FASE 4: Reportes (PUEDE ESPERAR)

13. Controller de Reportes
14. Vistas de reportes

NOTAS IMPORTANTES

Sobre los Modelos Consolidados:

El archivo `modelos_ventas.java` contiene varias clases. Debes:

1. Copiar cada clase a su propio archivo
2. Cambiar `class` por `public class`
3. Asegurarte que cada archivo tenga el package correcto

Sobre los DAOs y Services Consolidados:

Similar a los modelos:

1. Extraer cada clase a su archivo
2. Hacer las clases públicas
3. Verificar imports

Imports Necesarios:

Todos los archivos necesitarán estos imports básicos:

```
java

package com.style.stock.model; // o .dao, .service, .controller

import java.time.LocalDateTime;
import java.time.LocalDate;
import java.time.LocalTime;
import java.util.List;
import java.util.ArrayList;
import java.util.Objects;
import java.util.Optional;
```



TESTING

Orden de Pruebas:

1. **Iniciar la aplicación** - Verificar que carga
2. **Probar menú** - Navegar entre secciones
3. **Crear categoría** - Ropa con atributos Talle y Color
4. **Crear producto** - Remera con precios mayorista/minorista
5. **Crear variantes** - Remera Talle 42 Negro, etc.
6. **Crear cliente** - Cliente minorista
7. **Realizar venta** - Venta minorista con ticket
8. **Verificar stock** - Comprobar que se descontó



TROUBLESHOOTING

Error: "Class not found"

- Verifica que el package sea correcto
- Verifica que la clase sea pública

- Limpia y recompila: `mvn clean install`

Error: "Cannot connect to database"

- Verifica que la ruta existe: `~/style-stock/`
- Dale permisos si es necesario
- Revisa los logs en `~/style-stock/logs/`

Error: "FXML not found"

- Verifica que el archivo esté en `src/main/resources/fxml/`
- Verifica que el path en el controller sea correcto: `/fxml/archivo.fxml`

Error en SQL:

- Revisa `~/style-stock/logs/stylestock.log`
- Verifica que el schema se haya creado correctamente

RESULTADO ESPERADO

Al finalizar tendrás:

- ✓ Sistema completo de ventas con precios mayorista/minorista
- ✓ Productos flexibles con atributos dinámicos
- ✓ Emisión de tickets y facturas A/B/C
- ✓ Pagos combinados con múltiples métodos
- ✓ Control de stock por variante
- ✓ Auditoría completa de movimientos
- ✓ Interfaz moderna y funcional

SIGUIENTE PASO

¿Qué necesitas que genere ahora?

1. **Controladores faltantes** (Producto, Variante, Cliente, etc.)
2. **Vistas FXML faltantes**
3. **DAOs/Services faltantes**
4. **Separar archivos consolidados**
5. **Otra cosa específica**

¡Dime por dónde seguimos! 🚀